

UPASS



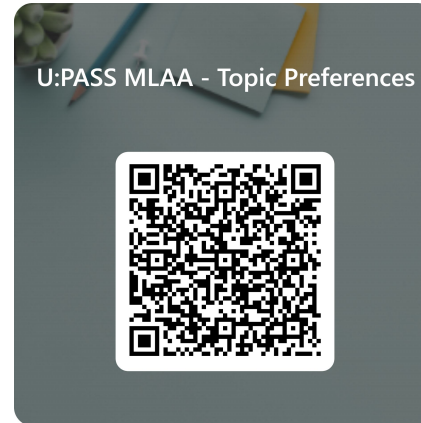
UPASS Sessions
with **Katherin Gomez Londono**

on

- **Thursdays 4-5pm CB01.05.01**
- **Fridays 4-5pm CB.08.03.005**

www.uts.edu.au/upass-sessions

You can mention what topics you want to cover next sessions by filling this form:



Machine Learning Algorithms and Applications

Lecture 3

(Block 1)

Plan for Today



Data Quality



K-Nearest Neighbors



Categorical Data



Lab



Data Quality

Data Quality Issues

Duplicates

Multiple records in the dataset represent the same single case

Missing Values

Specific observations or features with missing value

Invalid Values

Values that don't fit the variable requirements (e.g. negative values for income)

Incorrect Format

Values with known coding schemes (e.g. dd/mm/yy versus mm/dd/yy)

Outliers

Variables with extreme values that are either very high or very low

Value Inconsistencies

Values between variables with known relationships are inconsistent (e.g. salary for unemployed people)

Identifying Duplicates

	date	id	average	spread	volume	premium
0	01/01/19	1	\$70	35	45	Yes
1	01/01/20	1	\$75	56	52	Yes
2	01/02/20	2	\$40	46	50	Yes
3	01/01/19	1	\$70	35	45	Yes
4	01/03/20	3	\$110	53	75	No

Find duplicates:

```
mask = df.duplicated()  
print(mask)
```



```
0    False  
1    False  
2    False  
3     True  
4    False  
dtype: bool
```

Extract
duplicates:

```
print(df[mask])
```



```
   date id average spread volume premium  
3 01/01/19 1    $70    35     45     Yes
```

<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.duplicated.html?highlight=duplicated#pandas.DataFrame.duplicated>

Cleaning Duplicates

	date	id	average	spread	volume	premium
0	01/01/19	1	\$70	35	45	Yes
1	01/01/20	1	\$75	56	52	Yes
2	01/02/20	2	\$40	46	50	Yes
3	01/01/19	1	\$70	35	45	Yes
4	01/03/20	3	\$110	53	75	No

Remove
duplicates:

```
df = df.drop_duplicates()  
df.drop_duplicates(inplace=True)
```



	date	id	average	spread	volume	premium
0	01/01/19	1	\$70	35	45	Yes
1	01/01/20	1	\$75	56	52	Yes
2	01/02/20	2	\$40	46	50	Yes
4	01/03/20	3	\$110	53	75	No

https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.drop_duplicates.html

Identifying Missing Values

	date	id	average	spread	volume	premium
0	01/01/19	1.0	\$70	35.0	45.0	Yes
1	01/01/20	NaN	\$75	56.0	52.0	Yes
2	01/02/20	2.0	\$40	NaN	50.0	Yes
3	01/01/20	4.0	\$80	35.0	45.0	Yes
4	01/03/20	3.0	\$110	53.0	NaN	No

Find missing
values:

`df.isna()`



	date	id	average	spread	volume	premium
0	False	False	False	False	False	False
1	False	True	False	False	False	False
2	False	False	False	True	False	False
3	False	False	False	False	False	False
4	False	False	False	False	True	False

<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.isna.html?highlight=isna#pandas.DataFrame.isna>

Cleaning Missing Values

	date	id	average	spread	volume	premium
0	01/01/19	1.0	\$70	35.0	45.0	Yes
1	01/01/20	NaN	\$75	56.0	52.0	Yes
2	01/02/20	2.0	\$40	NaN	50.0	Yes
3	01/01/20	4.0	\$80	35.0	45.0	Yes
4	01/03/20	3.0	\$110	53.0	NaN	No

Remove missing values:

```
df = df.dropna()
df.dropna(inplace=True)
```



	date	id	average	spread	volume	premium
0	01/01/19	1.0	\$70	35.0	45.0	Yes
3	01/01/20	4.0	\$80	35.0	45.0	Yes

<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.isna.html?highlight=isna#pandas.DataFrame.isna>

Replace missing values:

```
df = df.fillna(0)
df.fillna(0, inplace=True)
```



	date	id	average	spread	volume	premium
0	01/01/19	1.0	\$70	35.0	45.0	Yes
1	01/01/20	0.0	\$75	56.0	52.0	Yes
2	01/02/20	2.0	\$40	0.0	50.0	Yes
3	01/01/20	4.0	\$80	35.0	45.0	Yes
4	01/03/20	3.0	\$110	53.0	0.0	No

<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.fillna.html?highlight=fillna#pandas.DataFrame.fillna>

Identifying Invalid/Incorrect/Outlier Values

	date	id	average	spread	volume	premium
0	01/01/19	1	\$70	35	45.00	Yes
1	01/01/20	1	\$75	56	52.00	Yes
2	01/02/20	2	\$40	45	6.89	Yes
3	01/01/20	4	\$80	35	45.00	Missing
4	01/03/20	3	-\$93	53	50.00	No

Find row with
starting string:

```
mask1 = df["average"].str.startswith("-")  
df[mask1]
```



	date	id	average	spread	volume	premium
4	01/03/20	3	-\$93	53	50.0	No

<https://pandas.pydata.org/docs/reference/api/pandas.Series.str.startswith.html>

Find row with
value within a
list:

```
mask3 = df["premium"].isin(["Missing"])  
df[mask3]
```



	date	id	average	spread	volume	premium
3	01/01/20	4	\$80	35	45.0	Missing

<https://pandas.pydata.org/docs/reference/api/pandas.Series.isin.html?highlight=isin#pandas.Series.isin>

Find row with
number below a
threshold:

```
mask2 = df["volume"] < 45  
df[mask2]
```



	date	id	average	spread	volume	premium
2	01/02/20	2	\$40	45	6.89	Yes

Cleaning Invalid/Incorrect/Outlier Values

	date	id	average	spread	volume	premium
0	01/01/19	1	\$70	35	45.00	Yes
1	01/01/20	1	\$75	56	52.00	Yes
2	01/02/20	2	\$40	45	6.89	Yes
3	01/01/20	4	\$80	35	45.00	Missing
4	01/03/20	3	-\$93	53	50.00	No

Replace with .loc:

```
mask1 = df["average"].str.startswith("-")  
df.loc[mask1, "average"] = "$93"
```



	date	id	average	spread	volume	premium
0	01/01/19	1	\$70	35	45.00	Yes
1	01/01/20	1	\$75	56	52.00	Yes
2	01/02/20	2	\$40	45	6.89	Yes
3	01/01/20	4	\$80	35	45.00	Missing
4	01/03/20	3	\$93	53	50.00	No

Replace with
.replace():

```
df["premium"].replace("Missing", value="No",  
inplace=True)
```



	date	id	average	spread	volume	premium
0	01/01/19	1	\$70	35	45.00	Yes
1	01/01/20	1	\$75	56	52.00	Yes
2	01/02/20	2	\$40	45	6.89	Yes
3	01/01/20	4	\$80	35	45.00	No
4	01/03/20	3	\$93	53	50.00	No

<https://pandas.pydata.org/docs/reference/api/pandas.Series.replace.html?highlight=replace#pandas.Series.replace>

Replace with
.where():

```
mask2 = df["volume"] < 45  
df["volume"].where(~mask2, other=45,  
inplace=True)
```



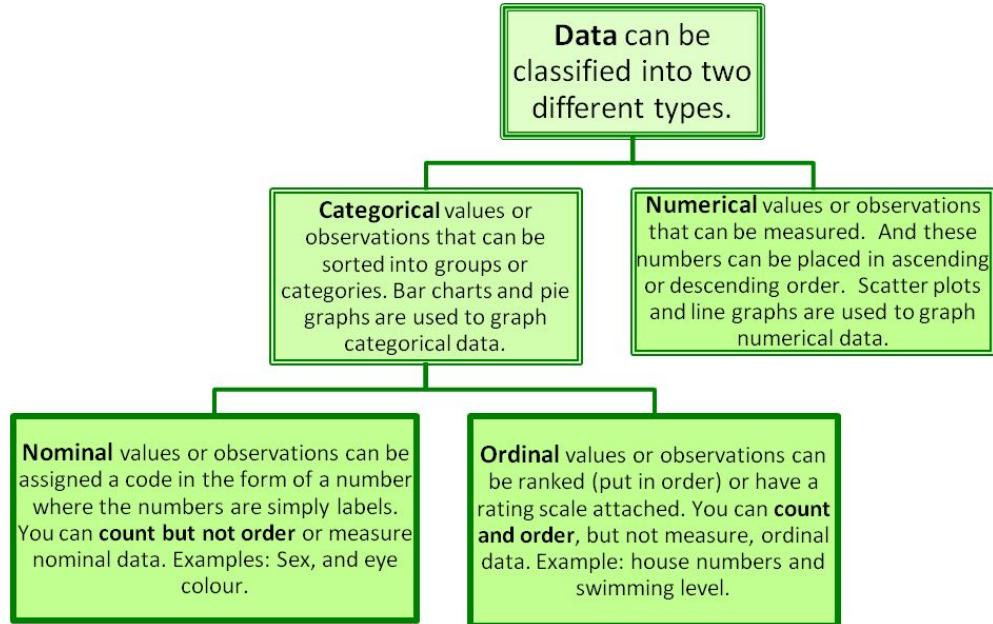
	date	id	average	spread	volume	premium
0	01/01/19	1	\$70	35	45.0	Yes
1	01/01/20	1	\$75	56	52.0	Yes
2	01/02/20	2	\$40	45	45.0	Yes
3	01/01/20	4	\$80	35	45.0	No
4	01/03/20	3	\$93	53	50.0	No

<https://pandas.pydata.org/docs/reference/api/pandas.Series.where.html?highlight=where#pandas.Series.where>



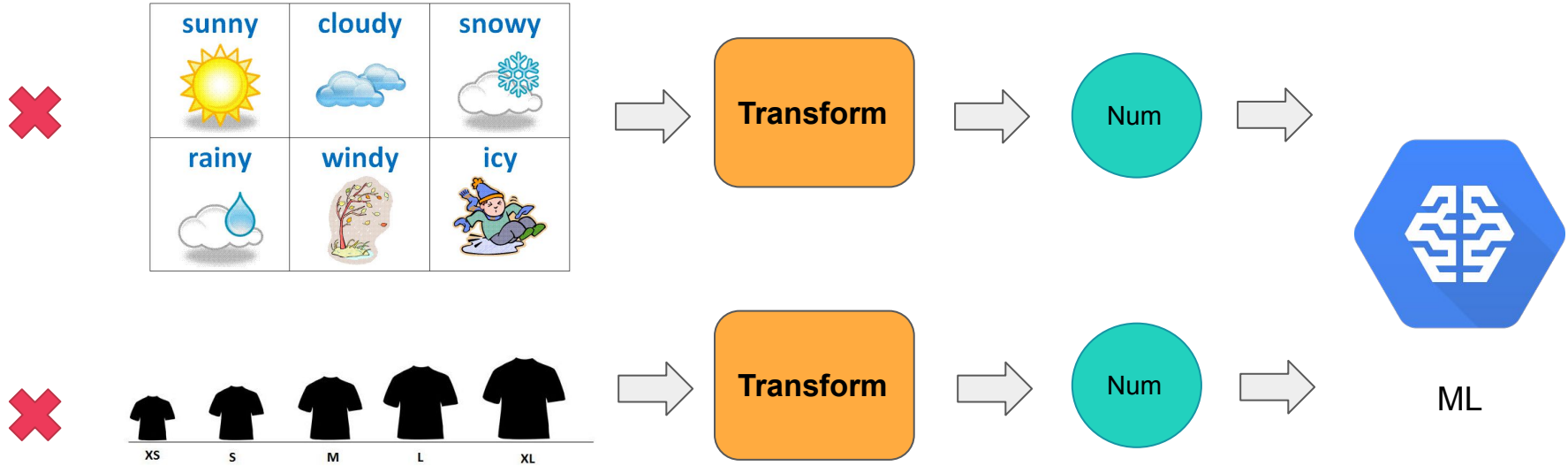
Categorical Data

Categorical Data



Machine Learning and Categorical Data

ML algorithms can not handle categorical data directly. They take only numerical values as inputs



Indexing

Objective:

Assign an index (integer) for each possible value of the categorical variable



risk	risk
low	1
low	1
medium	2
high	0
medium	2

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html?highlight=label%20encoder#sklearn.preprocessing.LabelEncoder>

```
from sklearn.preprocessing import LabelEncoder
```

```
risk = ["low", "low", "medium", "high", "medium"]
```

```
gle = LabelEncoder()
```

```
gle.fit_transform(risk)
```

```
array([1, 1, 2, 0, 2])
```


Indexing with Mapping

Objective:

Assign an index (integer) for each possible value of the categorical variable with a specified order

risk		risk
low	{ 'high': 2, 'low': 0, 'medium': 1 }	0
low		0
medium	➔	1
high		2
medium		1

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OrdinalEncoder.html#sklearn.preprocessing.OrdinalEncoder>

```
from sklearn.preprocessing import OrdinalEncoder

risk = [ ["low"], ["low"], ["medium"], ["high"], ["medium"] ]

ole = OrdinalEncoder(categories=[ ['low', 'medium', 'high'] ])

ole.fit_transform(risk)

array([[0.],
       [0.],
       [1.],
       [2.],
       [1.]])
```


One-Hot Encoding

Objective:

Assign a column for each value with binary outcome

Color	
red	
red	
yellow	
green	
yellow	



green	red	yellow
0	1	0
0	1	0
0	0	1
1	0	0
0	0	1

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html#sklearn.preprocessing.OneHotEncoder>

```
from sklearn.preprocessing import OneHotEncoder

color = [{"red"}, {"red"}, {"yellow"}, {"green"}, {"yellow"}]

ohe = OneHotEncoder()

ohe.fit_transform(color).toarray()

array([[0., 1., 0.],
       [0., 1., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 0., 1.]])
```



K-Nearest Neighbors

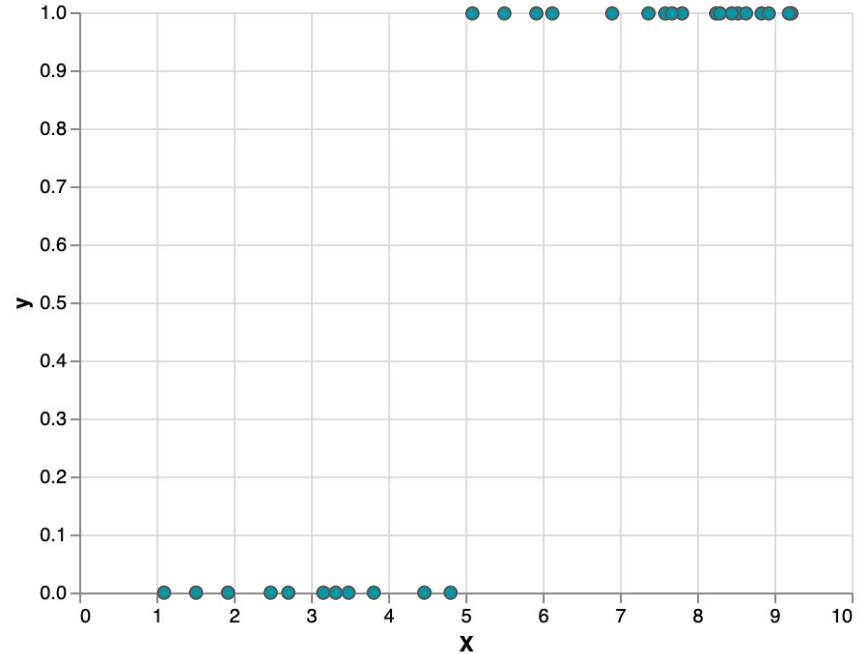
K-nearest neighbors

Nearest Neighbors:

Use observations' proximity to determine correct prediction

K:

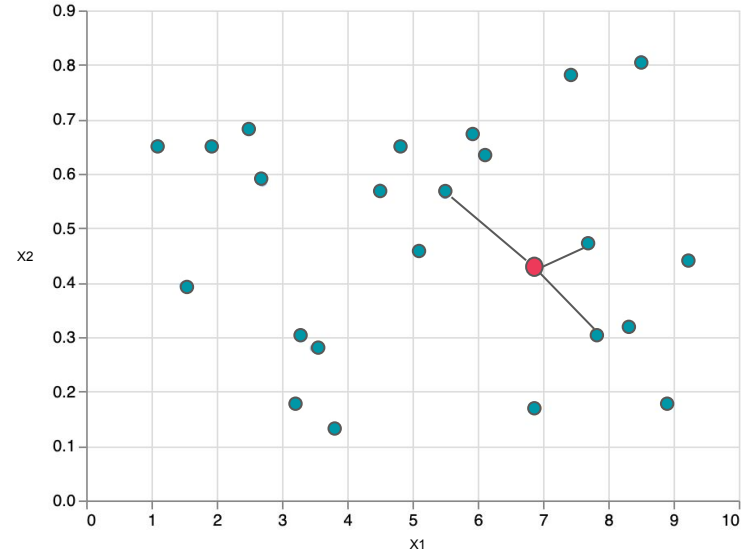
How many neighbor the model will consider



Note: KNN can be used for regression and classification

K-nearest neighbors Algorithm

1. Load the training data
2. Choose the value of K
3. Load the observation to predict on
4. Calculate the Euclidean distance between the observation and all training data points
5. Store the Euclidean distances in a list and sort it
6. Subset the first K points
7. Assign a class to the observation based on the majority of classes present in the chosen points



If $K = 3$, then observation will be the average of the 3 nearest neighbors.

Scikit-learn KNN

Generic API for sklearn models:



Import class

```
from sklearn.neighbors import NearestNeighbors
```



Instantiate a model with hyperparameter

```
knn = NearestNeighbors(n_neighbors=5)
```



Fit a model

```
knn.fit(X, y)
```



Predict with a model

```
knn.predict(new_X)
```

Sklearn Parameters

<https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.NearestNeighbors.html?highlight=nearest%20neighbor#sklearn.neighbors.NearestNeighbors>

n_neighbors : int, default=5

Number of neighbors to use by default for `kneighbors` queries.

radius : float, default=1.0

Range of parameter space to use by default for `radius_neighbors` queries.

algorithm : {'auto', 'ball_tree', 'kd_tree', 'brute'}, default='auto'

Algorithm used to compute the nearest neighbors:

- 'ball_tree' will use `BallTree`
- 'kd_tree' will use `KDTree`
- 'brute' will use a brute-force search.
- 'auto' will attempt to decide the most appropriate algorithm based on the values passed to `fit` method.

Note: fitting on sparse input will override the setting of this parameter, using brute force.

leaf_size : int, default=30

Leaf size passed to `BallTree` or `KDTree`. This can affect the speed of the construction and query, as well as the memory required to store the tree. The optimal value depends on the nature of the problem.

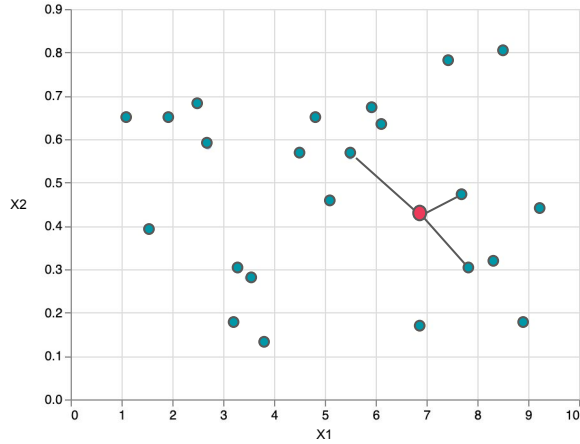
metric : str or callable, default='minkowski'

the distance metric to use for the tree. The default metric is `minkowski`, and with $p=2$ is equivalent to the standard Euclidean metric. See the documentation of `DistanceMetric` for a list of available metrics. If metric is "precomputed", X is assumed to be a distance matrix and must be square during fit. X may be a `sparse graph`, in which case only "nonzero" elements may be considered neighbors.

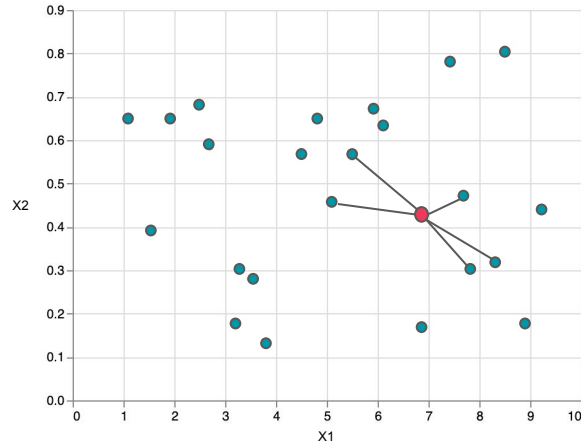
p : int, default=2

Parameter for the Minkowski metric from `sklearn.metrics.pairwise.pairwise_distances`. When $p = 1$, this is equivalent to using `manhattan_distance` (l1), and `euclidean_distance` (l2) for $p = 2$. For arbitrary p , `minkowski_distance` (l_p) is used.

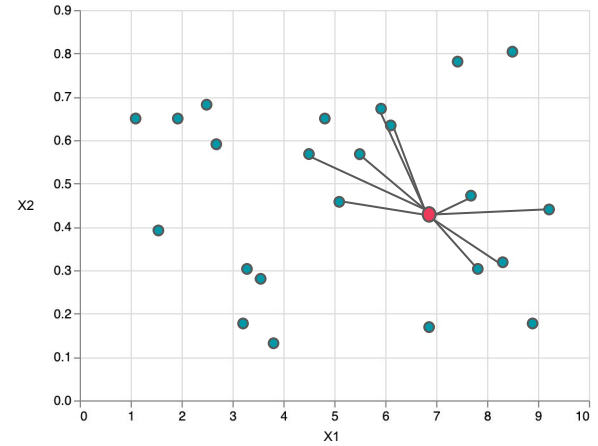
Fitting a KNN



k = 3

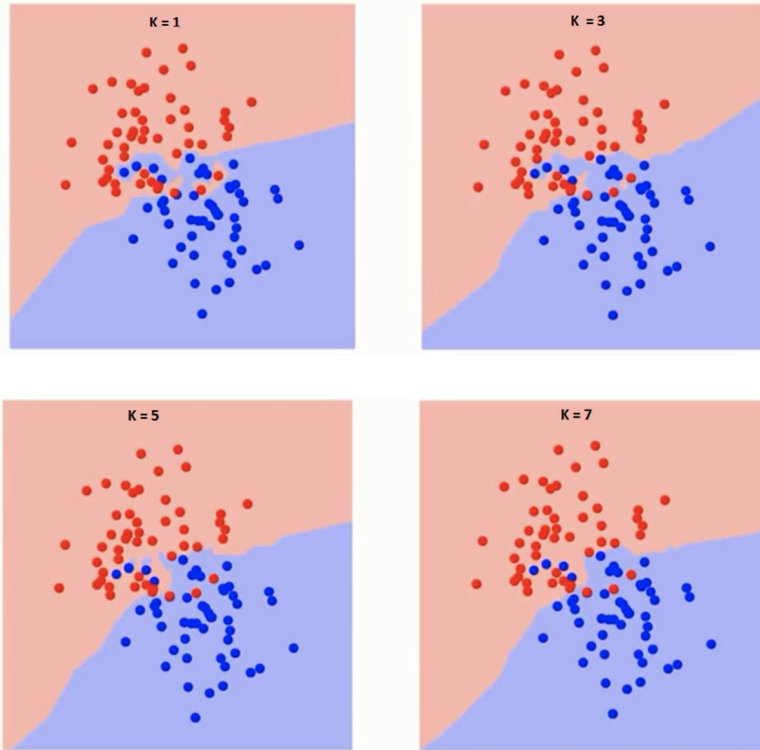


k = 5



k = 9

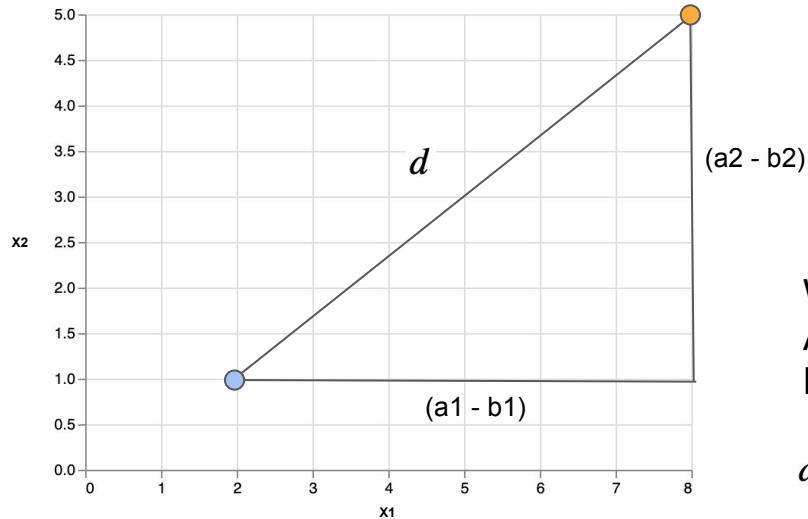
Influence of k in KNN



Higher value of k leads to smoother boundaries and potentially better generalisation

Lower value of k leads to very specific boundaries and potentially overfitting

Euclidean Distance



Formula:

$$\text{distance}(a, b) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

With:

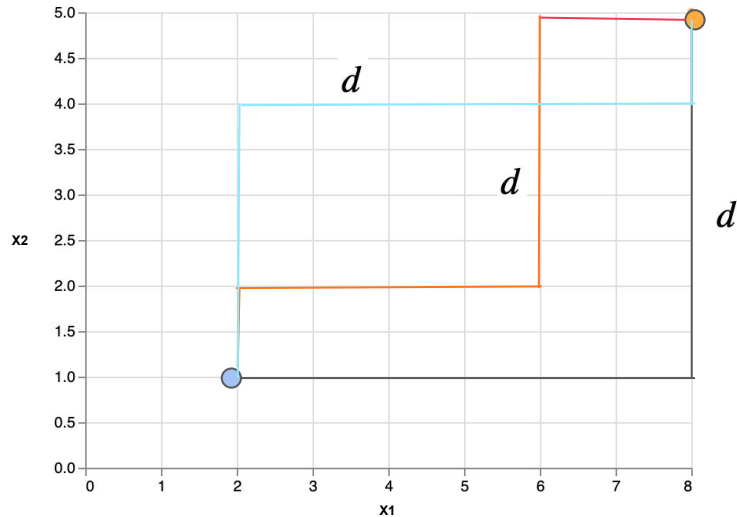
$$A = (2, 1)$$

$$B = (8, 5)$$

$$d^2(A, B) = (2 - 8)^2 + (1 - 5)^2 = (-6)^2 + (-4)^2 = 36 + 16 = 52$$

$$d(A, B) = \sqrt{52} = 7.2$$

Manhattan Distance



Formula:

$$\text{distance}(a, b) = \sum_{i=1}^n |a_i - b_i|$$

With:

A = (2, 1)

B = (8, 5)

$$d(A, B) = |2 - 8| + |1 - 5| = |-6| + |-4| = 6 + 4 = 10$$



Time for Lab

